LECTURE 07

Cloud Storage and Distributed File Systems

Object, Block & File Storage • GFS, HDFS & Amazon S3 • AWS Storage Deep Dive • Design Trade-offs

Agenda

1. **Storage Fundamentals**
   Object, block, and file storage models
2. **Distributed FS Case Studies**
   Google File System (GFS) and HDFS
3. **Amazon S3 Deep Dive**
   Architecture, durability, storage classes
4. **AWS Storage Ecosystem**
   EBS, EFS, and the broader AWS toolkit
5. **Design Trade-offs**
   Consistency, replication, and cost
6. **Key Takeaways**
   Bringing it all together

Why Cloud Storage Matters

Data volumes, durability requirements, and elastic compute all push storage design in the same direction: decoupled, distributed, and fault-tolerant.

181 ZB

Global data created/replicated by 2025 (IDC est.)

99.999999999%

Amazon S3's designed object durability (11 nines)

Exabytes

Scale distributed file systems like GFS were built to reach

Three forces driving distributed storage design:

- Scale — single machines cannot hold or serve modern datasets
- Durability — hardware fails constantly at scale, so redundancy must be automatic
- Elasticity — storage must grow/shrink independently of compute

Storage Fundamentals

Object, block, and file storage models

Three Cloud Storage Models

Each model trades off access granularity, latency, and scale differently.

B

Block Storage

Fixed-size blocks, raw-disk semantics

EXAMPLES

EBS, Persistent Disk

F

File Storage

Hierarchical files via NFS/SMB

EXAMPLES

EFS, Azure Files

O

Object Storage

Flat namespace, key + metadata via REST

EXAMPLES

S3, GCS, Blob

B Block Storage

Raw, addressable blocks — the closest cloud analog to a physical disk

HOW IT WORKS

- Data split into fixed-size blocks, each independently addressable
- OS or app layers a filesystem on top (ext4, NTFS, XFS)
- Supports random reads/writes with very low latency
- Typically attached to a single VM/instance at a time (AZ-bound)

USE CASES & EXAMPLES

- Best for: databases, boot volumes, transactional workloads
- High IOPS, low latency, predictable performance
- Examples: AWS EBS, Google Persistent Disk, Azure Managed Disks
- Scales per-volume, not automatically across a fleet

F

File Storage

A shared, hierarchical namespace accessed like a network drive

HOW IT WORKS

- Organized as directories and files, not raw blocks
- Accessed via standard protocols: NFS, SMB/CIFS
- POSIX-like semantics: permissions, file locking
- Multiple clients can mount and access concurrently

USE CASES & EXAMPLES

- Best for: shared home directories, CMS, lift-and-shift apps
- Simplifies migration of legacy file-based applications
- Examples: AWS EFS, Azure Files, Google Filestore
- Scales more elastically than block, less than object

0

Object Storage

Flat namespace of immutable objects, addressed by key over HTTP

HOW IT WORKS

- Each object = data + rich metadata + a unique key
- No true folder hierarchy — prefixes simulate structure
- Accessed via REST APIs (HTTP GET/PUT/DELETE)
- Objects are immutable — updates rewrite the whole object

USE CASES & EXAMPLES

- Best for: backups, static assets, data lakes, media archives
- Virtually unlimited scale — exabytes, no capacity planning
- Examples: Amazon S3, Google Cloud Storage, Azure Blob
- Higher per-request latency than block storage

Storage Types at a Glance

	Block	File	Object
Access unit	Block	File / directory	Whole object
Latency	Lowest	Low – medium	Higher
Scalability	Limited (per volume)	Medium	Massive (exabyte)
Metadata	Minimal	Filesystem attrs	Rich & custom
Best for	Databases, VMs	Shared file access	Unstructured, large-scale

Takeaway: choose block for low-latency structured I/O, file for shared hierarchical access, object for massive-scale unstructured data.

Distributed File System Case Studies

Google File System (GFS) and Hadoop Distributed File System (HDFS)

G

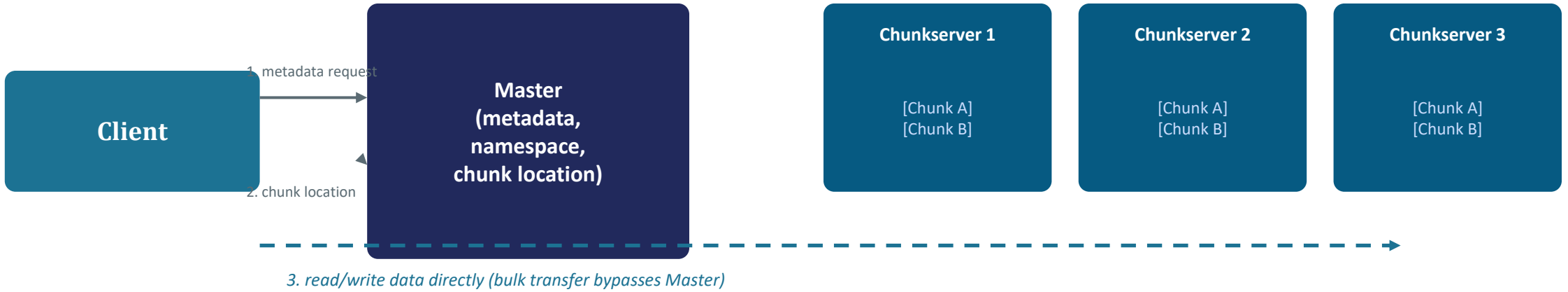
Google File System (GFS)

A distributed filesystem built for Google's internal large-scale batch workloads

DESIGN PHILOSOPHY	Assumes hardware failure is the norm, not the exception — reliability comes from software-level replication, not fault-tolerant hardware.
WORKLOAD FIT	Optimized for large files (100MB+) with sequential reads and append-heavy writes — not general-purpose small-file access.
CONSISTENCY	Deliberately relaxed / weak consistency model, traded for higher throughput on Google's specific workloads (crawling, indexing, MapReduce I/O).
LEGACY	Its master/chunkserver design directly inspired HDFS and much of the distributed storage systems that followed.

GFS Architecture

A single Master coordinates metadata; Chunkservers store replicated 64MB chunks



Replication: Each 64MB chunk is replicated 3× by default across Chunkservers (often across racks) so the loss of any single machine never loses data.

Key design choice: separating the metadata path (through the Master) from the data path (direct client-to-chunkserver) keeps the Master from becoming a throughput bottleneck, even though it remains a single point of coordination.

GFS: Consistency Model & Limitations

CONSISTENCY MODEL

- "Relaxed" rather than strict — favors throughput over strict correctness guarantees
- Concurrent writes to the same region may leave it "consistent but undefined"
- Record appends are at-least-once — duplicates and padding are possible
- Applications are expected to tolerate this (e.g., via checksums, unique IDs)

KEY LIMITATIONS

- Single Master is a metadata bottleneck and historical single point of failure
- Poor fit for workloads with many small files (Master memory pressure)
- Not designed for low-latency random writes or interactive applications
- Tightly coupled to Google's internal workloads — never released as a product

Hadoop Distributed File System (HDFS)

An open-source filesystem heavily inspired by GFS, built for the Hadoop ecosystem

HOW IT WORKS

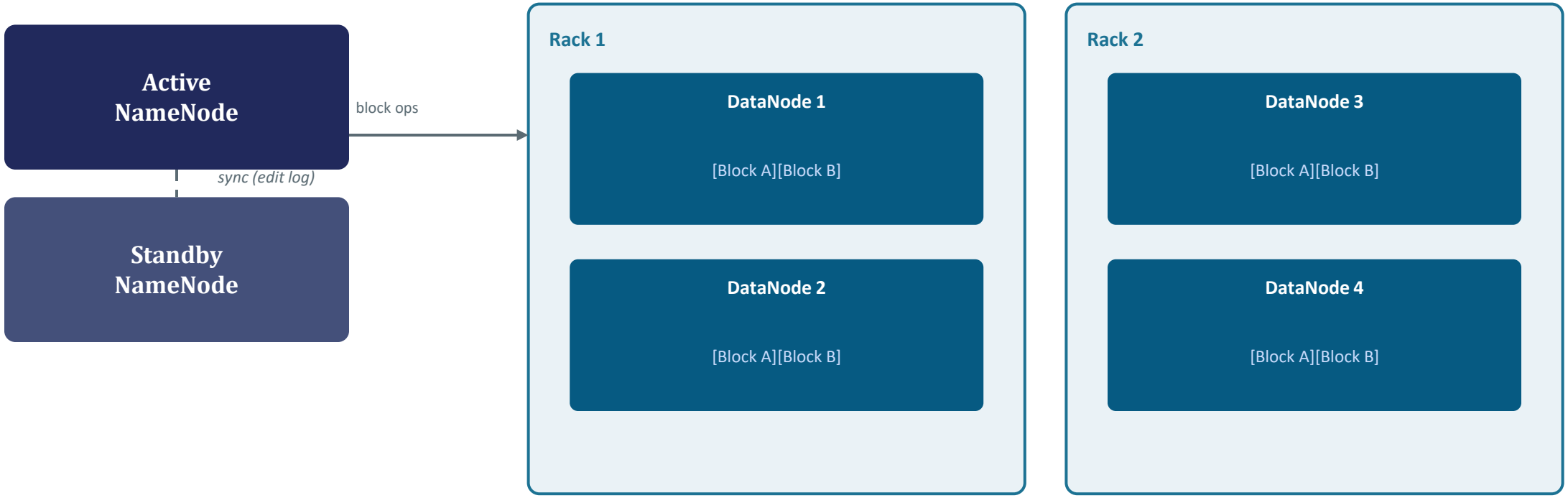
- Originally built to support MapReduce; now underlies Spark, Hive, and more
- Open-source and widely deployed outside of any single company
- Runs on commodity hardware clusters, on-prem or in the cloud
- Part of the broader Apache Hadoop project ecosystem

ARCHITECTURE AT A GLANCE

- NameNode holds metadata; DataNodes store actual data blocks
- Default block size 128MB (vs. GFS's 64MB)
- Blocks replicated 3× by default across DataNodes
- High-Availability NameNode pairs address the original SPOF

HDFS Architecture

Active/Standby NameNodes for high availability, DataNodes replicated with rack awareness



Rack awareness: HDFS deliberately places replicas across racks (e.g., 2 copies on one rack, 1 on another) so a single rack or top-of-rack switch failure never causes data loss — a refinement beyond GFS's original design.

HDFS: Features & the Small-File Problem

Write-once, read-many

Optimized for batch analytics (MapReduce, Spark, Hive) rather than transactional updates.

128MB default block size

Large blocks minimize metadata overhead and suit sequential scans over huge datasets.

Erasure coding (HDFS 3.x)

Reed-Solomon coding cuts storage overhead to ~1.5× instead of 3× full replication, at extra CPU cost to rebuild.

Small-file problem

Every file/block's metadata lives in NameNode RAM — millions of small files can exhaust memory and hurt performance.

GFS vs. HDFS

	GFS	HDFS
Origin	Internal Google system (2003 paper)	Open-source, Apache Hadoop project
Metadata node	Single Master	NameNode (HA Active/Standby pairs)
Default chunk/block size	64 MB	128 MB
Replication	3× (default)	3× default, or erasure coding (3.x)
Placement awareness	Basic	Explicit rack awareness
Primary workload	Google internal batch (search, indexing)	General-purpose Hadoop ecosystem (Spark, Hive)

S3

Amazon S3: Overview

A managed object storage service — not a traditional distributed filesystem

Bucket

A top-level container for objects; globally unique name, bound to an AWS region.

Object

The unit of storage — data plus metadata, addressed by a key inside a bucket.

Key

The unique identifier for an object within a bucket (S3 uses a flat namespace; "folders" are just key prefixes).

REST API

All access — from the console, CLI, or SDKs — flows through HTTP(S) API calls (GET/PUT/DELETE/LIST).

S3 Durability & Replication

S3 automatically spreads and replicates data across multiple facilities within a region

99.999999999%

Designed durability ("11 nines") for objects across a given year

Multi-AZ by default

Objects stored redundantly across multiple Availability Zones in the chosen region

No server management

Replication, hardware failures, and scaling are handled internally — users never provision nodes

Additional durability & resilience features:

- Versioning — preserves every version of an object, protecting against accidental overwrite/delete
- Cross-Region Replication (CRR) — asynchronously copies objects to a bucket in another region
- Object Lock — WORM protection for compliance (retention periods, legal holds)
- Multi-AZ independence — an entire Availability Zone can fail without data loss

S3 Storage Classes

Choose a class based on access frequency and retrieval-time tolerance to control cost

Storage Class	Access Pattern	Retrieval Time	Relative Cost
S3 Standard	Frequent access	Milliseconds	Highest
S3 Intelligent-Tiering	Unknown/changing	Milliseconds	Auto-optimized
S3 Standard-IA	Infrequent access	Milliseconds	Lower storage, retrieval fee
S3 One Zone-IA	Infrequent, re-creatable	Milliseconds	Lower (single AZ)
Glacier Instant Retrieval	Rarely accessed archives	Milliseconds	Very low
Glacier Flexible Retrieval	Archival	Minutes – hours	Very low
Glacier Deep Archive	Long-term compliance archive	Hours (up to 12h)	Lowest

Lifecycle policies automate transitions between classes as objects age.

S3 Consistency & Access Control

CONSISTENCY MODEL

- Strong read-after-write consistency for all requests (since Dec 2020)
- Applies to new object PUTs, overwrite PUTs, and DELETEs alike
- Replaced the earlier eventual-consistency model for overwrites/deletes
- Simplifies application logic — no need to handle stale reads after a write

SECURITY & ACCESS CONTROL

- IAM policies control which users/roles can act on which buckets
- Bucket policies set resource-level rules (e.g., allow/deny by IP, VPC)
- Encryption at rest: SSE-S3, SSE-KMS, or customer-provided keys (SSE-C)
- Block Public Access settings guard against accidental public exposure

PART 3

AWS Storage Ecosystem Deep Dive

Beyond S3: EBS, EFS, and the full AWS storage toolkit

The AWS Storage Landscape

AWS offers a purpose-built service for nearly every storage access pattern

EBS

EBS

Block

Persistent block volumes for EC2 instances

EFS

EFS

File

Elastic NFS file system, multi-AZ

S3

S3

Object

Scalable object storage, 11 nines durability

Gl

Glacier

Archive

Low-cost long-term archival (an S3 class)

FSx

FSx

File

Managed Windows/Lustre/NetApp/OpenZFS file systems

St

Storage Gateway

Hybrid

Bridges on-prem environments to AWS storage

Sn

Snow Family

Transfer

Physical devices for offline bulk data migration

Ba

Backup

Mgmt

Centralized backup policy across AWS services

EBS

AWS EBS: Elastic Block Store

Persistent block-level storage volumes attached to EC2 instances

Volume Type	Best For	Key Spec
gp3 (General Purpose SSD)	Boot volumes, dev/test, most workloads	3,000 IOPS / 125 MB/s baseline, scalable independently
io2 Block Express	High-performance databases (Oracle, SAP)	Up to 256,000 IOPS, sub-ms latency, 99.999% durability
st1 (Throughput HDD)	Big data, log processing, streaming	Low-cost, high throughput, not for random I/O
sc1 (Cold HDD)	Infrequently accessed data	Lowest cost per GB among EBS volumes

Other key EBS features:

- Bound to a single Availability Zone — snapshot to move across AZs/regions
- Point-in-time snapshots stored durably in S3, incremental after the first
- Can be resized, and volume type changed, without downtime

EFS

AWS EFS: Elastic File System

Fully managed, elastic NFSv4 file system shared across many instances and AZs

STORAGE CLASSES

- EFS Standard — frequently accessed files, multi-AZ resilience
- EFS Standard-IA — automatically tiers infrequently accessed files, lower cost
- EFS One Zone / One Zone-IA — single-AZ variants at reduced cost
- Lifecycle management moves files between classes automatically

THROUGHPUT & SCALE

- Bursting mode — throughput scales with storage size, with burst credits
- Provisioned mode — set a fixed throughput regardless of storage size
- Elastic mode — automatically scales throughput up/down with workload
- Grows/shrinks automatically as files are added or removed — no capacity planning

AWS Storage Choice: S3 vs. EBS vs. EFS

	Amazon S3	Amazon EBS	Amazon EFS
Storage type	Object	Block	File
Attach point	Accessed over network/API	One EC2 instance at a time*	Many instances, multi-AZ
Scalability	Virtually unlimited	Fixed size (resizable)	Elastic, grows automatically
Latency	Higher (network/API calls)	Lowest (near-local disk)	Low-medium (network FS)
Typical use	Backups, data lakes, static assets	Databases, boot volumes	Shared content, home dirs
Durability model	11 nines, multi-AZ automatic	Single AZ; snapshots to S3	Multi-AZ (Standard class)

**io2 Block Express Multi-Attach allows a single io2 volume to attach to multiple instances in limited scenarios.*

Distributed File System Design Trade-offs

Every design decision below trades one property for another — there is no free lunch

1 Consistency vs. Availability

Strong consistency costs latency/availability during partitions; eventual consistency improves availability but risks stale reads (GFS/HDFS relaxed; S3 strong).

2 Metadata: Centralized vs. Distributed

A single master (GFS) is simple but a scaling/SPOF risk; distributed metadata (Ceph, HDFS Federation) scales further but needs consensus protocols (Paxos/Raft).

3 Replication Strategy

Full 3× replication is simple and fast to recover but costly; erasure coding cuts storage to ~1.5× at the cost of CPU-heavy rebuilds.

4 Block Size & Access Pattern

Large blocks (64–128MB) suit sequential big-file workloads; small blocks suit random access but blow up metadata overhead.

Key Takeaways

1. Match storage type to access pattern: block for low-latency structured I/O, file for shared hierarchical access, object for massive-scale unstructured data.

2. GFS and HDFS pioneered the master/worker distributed filesystem pattern with relaxed consistency, optimized for large sequential batch workloads.

3. Amazon S3 is a different design point entirely — a managed object storage service trading filesystem semantics for massive scale and near-zero operational overhead.

4. AWS pairs S3 with EBS (block) and EFS (file) to cover every access pattern, each with its own durability, scaling, and cost model.

5. Distributed storage design is a continuous set of trade-offs: consistency vs. availability, replication cost vs. durability, and latency vs. throughput.